Министерство науки и высшего образования РФ
Пермский национальный исследовательский политехнический университет
Электротехнический факультет
Кафедра Информационные технологии и автоматизированные системы

Базы данных

Лабораторная работа № 4

Тема: «Реализация команд DDL, DML и соединений на веб-странице»

Выполнил: студент
группы РИС-25-2б
Ильинов Ф. М.
Проверил: доцент кафедры ИТАС
Петренко А.А.

г. Пермь – 2026

СОДЕРЖАНИЕ

1 Цель работы

Реализовать веб-приложение для взаимодействия с СУБД PostgreSQL, обеспечивающее выполнение полного жизненного цикла операций с реляционной базой данных:

а) Определение структуры данных (команды DDL: создание, модификация, очистка и удаление таблиц);

б) Манипулирование данными (команды DML: вставка, обновление и удаление кортежей);

в) Выборка данных и реляционное соединение (операторы DQL: фильтрация SELECT ... WHERE и внутреннее соединение INNER JOIN ... ON);

г) Интроспекция схемы через системный каталог `information_schema` с динамическим формированием интерфейса и отображением сгенерированных SQL-запросов на странице браузера.

2 Задачи работы

В соответствии с целью были поставлены следующие задачи:

а) Разработать локальный веб-сервер на языке Golang (версия 1.27) с использованием стандартной библиотеки `net/http`;

б) Реализовать подключение к СУБД PostgreSQL с использованием пула соединений `pgx`;

в) Разработать механизм безопасной трансляции параметров пользовательских форм в команды SQL с экранированием идентификаторов и литералов;

г) Реализовать стек вызовов для команд DDL: двухэтапное создание таблиц с поддержкой первичных и внешних ключей (включая самоссылающиеся ограничения), изменение структуры через ALTER TABLE, очистку через TRUNCATE и каскадное удаление через DROP TABLE CASCADE;

д) Реализовать стек вызовов для команд DML: пакетную вставку записей с обработкой NULL-значений, модификацию данных через UPDATE и удаление данных через DELETE с контролем количества затронутых строк;

е) Реализовать стек вызовов для выборок и соединений: построитель запросов SELECT с предикатами фильтрации и оператор соединения INNER JOIN по ключевым полям;

ж) Реализовать опрос метаданных реляционной схемы через системные представления `information_schema.tables` и `information_schema.columns`;

и) Разработать веб-интерфейс на основе Server-Side Rendering (пакет `html/template`) с выводом результатов и текста выполненного SQL-запроса на страницу браузера.

3 Этапы выполнения

Для выполнения поставленных задач были сформулированы следующие технологические требования к инструментарию:

– Поддержка реляционной модели данных и стандарта SQL в СУБД PostgreSQL [**?**, **?**];

– Поддержка сетевого протокола передачи гипертекста HTTP [**?**];

– Обеспечение безопасности при формировании SQL-запросов и защита от SQL-инъекций;

– Возможность интроспекции структуры базы данных на основе системного каталога;

– Возможность отрисовки форм, кнопок и динамических таблиц в веб-интерфейсе.

Для реализации клиентского интерфейса был выбран язык разметки HTML [**?**] совместно с библиотекой стилей Pico CSS [**?**], обеспечивающей лаконичное визуальное оформление без необходимости написания вспомогательного клиентского JavaScript-кода.

В качестве языка серверной части выбран Golang версии 1.27 [**?**]. Для взаимодействия с СУБД PostgreSQL задействован драйвер и пул соединений `github.com/jackc/pgx` [**?**].

Сервер функционирует по принципу Server-Side Rendering (SSR). Все страницы собираются на стороне сервера с помощью встроенного шаблонизатора `html/template` и возвращаются браузеру в виде готового HTML-документа. Для изоляции мутирующих операций DDL и DML от данных других лабораторных работ выделена отдельная база данных `lab04_ilinov`.

3.1 Архитектура веб-приложения и стек вызовов

Веб-приложение построено по многоуровневой схеме. Прохождение любого пользовательского действия организовано в виде строгого сквозного стека вызовов (рис. **??**):

а) **Клиентский уровень**: веб-страница браузера (HTML-формы с параметрами операций DDL, DML или DQL);

б) **Уровень веб-сервера и маршрутизации**: маршрутизатор стандартной библиотеки Golang (версия 1.27), диспетчеризация по HTTP-методу и пути, распаковка параметров формы;

в) **Уровень сборщика запросов (Query Builder)**: санитизация параметров, экранирование идентификаторов PostgreSQL двойными кавычками (`quoteIdent`) и строковых литералов одинарными кавычками (`quoteLiteral`);

г) **Уровень СУБД PostgreSQL**: выполнение запроса через пул соединений `pgx`, транзакции, опрос системного каталога `information_schema`;

д) **Уровень представления (View Renderer)**: динамическое сканирование кортежей и отрисовка HTML-шаблона с отображением текста выполненного SQL-запроса и полученных данных.

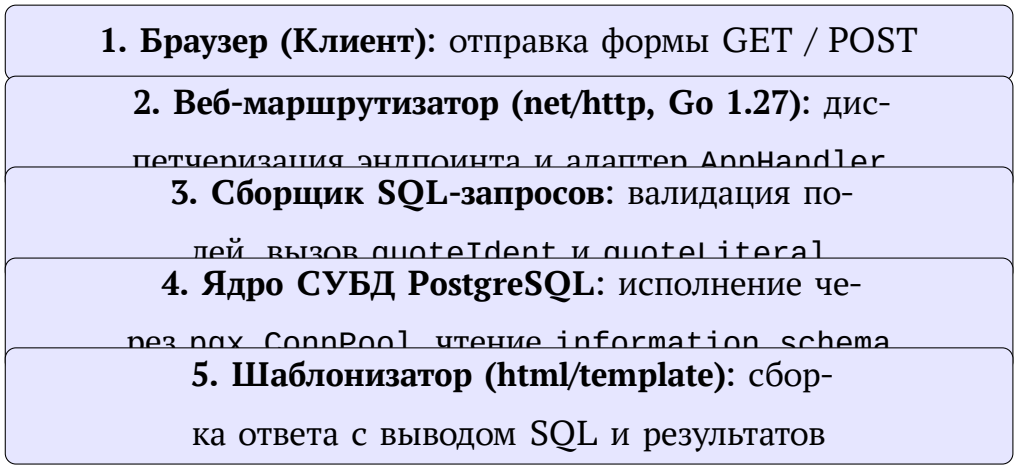


Рисунок 3.1 — Сквозной стек вызовов обработки запроса

3.1.1 Веб-сервер и маршрутизация запросов

Инициализация сетевого мультиплексора и регистрация маршрутов выполняются в функции `main` (листинг **??**). Начиная с версии Go 1.22 и актуальной в проекте Go 1.27, маршрутизатор `http.ServeMux` поддерживает декларацию HTTP-методов и параметров в строке шаблона.

```
74                http.Redirect(w, r, "/tables", http.StatusSeeOther)
75        })
76
77        mux.Handle("GET /tables", HandleTables(views, pool))
78        mux.Handle("GET /tables/{name}", HandleTables(views, pool))
79
80        mux.HandleFunc("GET /ddl", func(w http.ResponseWriter, r
          ↪  *http.Request) {
81                http.Redirect(w, r, "/ddl/create", http.StatusSeeOther)
82        })
83        mux.Handle("GET /ddl/{op}", HandleDDL(views, pool))
84        mux.Handle("POST /ddl/create", CreateTableHandler(views, pool))
85        mux.Handle("POST /ddl/alter", AlterTableHandler(views, pool))
86        mux.Handle("POST /ddl/truncate", TruncateTableHandler(views, pool))
87        mux.Handle("POST /ddl/drop", DropTableHandler(views, pool))
88
89        mux.HandleFunc("GET /dml", func(w http.ResponseWriter, r
          ↪  *http.Request) {
90                http.Redirect(w, r, "/dml/insert", http.StatusSeeOther)
91        })
92        mux.Handle("GET /dml/{op}", HandleDML(views, pool))
93        mux.Handle("POST /dml/insert", InsertHandler(views, pool))
94        mux.Handle("POST /dml/update", UpdateHandler(views, pool))
95        mux.Handle("POST /dml/delete", DeleteHandler(views, pool))
96
97        mux.Handle("GET /select", HandleSelect(views, pool))
98        mux.Handle("GET /select/{table}", HandleSelect(views, pool))
```

Листинг 1 — Маршрутизация HTTP-запросов DDL, DML и DQL в `main.go`

В строке 69 шаблон `"GET /{$}"` задает строгое соответствие корневому пути и выполняет перенаправление на страницу таблиц `/tables`. Для остальных операций предусмотрено четкое разделение:

— `GET`-запросы отображают страницы и формы (например, `/ddl/{op}`, `/dml/{op}`, `/select`);

— POST-запросы принимают данные форм и запускают сборку и выполнение соответствующих SQL-команд (/ddl/create, /dml/insert, /query/select и др.).

3.1.2 Архитектура обработчиков и обработка ошибок

Для унификации обработки ошибок используется тип-адаптер AppHandler (листинг **??**), представляющий собой функцию, возвращающую ошибку.

```
141         if e.Err != nil {
142                 return e.Err.Error()
143         }
144         return "Internal Server Error"
145 }
146
147 func (e *AppError) Unwrap() error {
148         return e.Err
149 }
150
151 type AppHandler func(w http.ResponseWriter, r *http.Request) error
152
153 func (fn AppHandler) ServeHTTP(w http.ResponseWriter, r *http.Request) {
154         if err := fn(w, r); err != nil {
155                 var appErr *AppError
156                 if errors.As(err, &appErr) {
157                         slog.Error("request error",
158                                 slog.String("path", r.URL.Path),
159                                 slog.String("message", appErr.Message),
160                                 slog.Any("err", appErr.Err),
161                         )
162                         code := appErr.Code
163                         if code == 0 {
```

Листинг 2 — Адаптер обработчиков AppHandler и структура AppError

Метод ServeHTTP перехватывает ошибки выполнения. Если возвращена ошибка AppError, фиксируется запись в логе и клиенту отправляется соответствующий HTTP-статус. При возникновении ошибки на уровне СУБД форма рендерится повторно со статусом 400 Bad Request, сохраняя введенные пользователем значения и подсвечивая сообщение сервера PostgreSQL и проблемный SQL-запрос.

3.1.3 Движок представления и шаблонизация

Генерация веб-интерфейса реализована на стороне сервера с помощью встроенного пакета `html/template` и обертки `ViewRenderer` (листинг **??**).

```
}

func NewViewRenderer(funcMap template.FuncMap) (*ViewRenderer, error) {
        layout, err :=
        ↪  template.New("layout.html").Funcs(funcMap).ParseFiles("templates/layout.h
        if err != nil {
                return nil, fmt.Errorf("parse layout.html: %w", err)
        }

        pages := []string{"tables", "ddl", "dml", "select", "join"}
        tmpls := make(map[string]*template.Template, len(pages))
        for _, p := range pages {
                t, err := layout.Clone()
                if err != nil {
                        return nil, fmt.Errorf("clone layout for %s: %w",
                        ↪  p, err)
                }
                path := filepath.Join("templates", p+".html")
                t, err = t.ParseFiles(path)
                if err != nil {
                        return nil, fmt.Errorf("parse %s: %w", path, err)
                }
                tmpls[p] = t
        }
        return &ViewRenderer{templates: tmpls}, nil
}

func (v *ViewRenderer) Render(w http.ResponseWriter, name string, data any)
↪  error {
        return v.RenderStatus(w, name, http.StatusOK, data)
}

func (v *ViewRenderer) RenderStatus(w http.ResponseWriter, name string,
↪  status int, data any) error {
        tmpl, ok := v.templates[name]
        if !ok {
                return fmt.Errorf("template %q not found", name)
```

Листинг 3 — Инициализация и рендеринг шаблонов в `views.go`

При запуске сервера компилируется базовый шаблон `layout.html`, после чего методом `Clone()` создаются изолированные копии для каждой рабочей страницы (`tables`, `ddl`, `dml`, `select`, `join`). Для динамических вычислений в разметке зарегистрированы специальные функции (листинг **??**).

```
41                os.Exit(1)
42        }
43
44        funcMap := template.FuncMap{
45                "add": func(a, b int) int { return a + b },
46                "seq": func(n int) []int {
47                        res := make([]int, n)
48                        for i := range res {
49                                res[i] = i
50                        }
51                        return res
52                },
53                "eqStr": func(a, b string) bool { return a == b },
54                "getRowVal": func(rows []map[string]string, rIdx int,
                  ↪  colName string) string {
55                        if rIdx >= 0 && rIdx < len(rows) && rows[rIdx] !=
                          ↪  nil {
56                                return rows[rIdx][colName]
```

Листинг 4 — Пользовательские функции шаблонизатора `funcMap`

Функции `seq` и `add` используются для циклической генерации динамических строк формы (например, добавление заданного количества колонок в DDL или кортежей в DML). Функция `getRowVal` обеспечивает безопасное извлечение сохраненных значений полей при повторной отрисовке после ошибок.

Базовый шаблон `layout.html` (листинг **??**) содержит навигацию, кнопку быстрого сброса БД и блок отображения сгенерированного SQL-запроса.

В строках 28–35 размещен блок уведомления об ошибках: если сервер СУБД возвращает ошибку синтаксиса или ограничений целостности, точный текст сформированного запроса выводится в теге `<pre><code>`, обеспечивая прозрачный аудит.

```
<main class="container">
    <header>
        <article>
            <strong>DB Laboratory Work №4</strong><br>
            <small>RIS-25-2b Ilinov F. M.</small>
            <form action="/reset" method="post">
                <button type="submit">Reset</button>
            </form>
        </article>
        <article>
            <a role="button" href="/tables">SHOW</a>
           <a role="button" href="/ddl/create">Data Definition Language</a>
            <a role="button" href="/dml/insert">Data Manipulation
            ↪  Language</a>
            <a role="button" href="/select">SELECT</a>
            <a role="button" href="/join">INNER JOIN</a>
        </article>
    </header>

    {{ if eqStr .Status "error" }}
    <article>
        <header><strong>ERROR:</strong> {{ .Message }}</header>
        {{ if .SQL }}
        <pre><code>{{ .SQL }}</code></pre>
        {{ end }}
    </article>
    {{ end }}

    {{ block "content" . }}
    {{ end }}

```

Листинг 5 — Разметка контейнера и блок аудита SQL в `layout.html`

3.1.4 Пул соединений и взаимодействие с PostgreSQL

Подключение к СУБД осуществляется в функции `main` через пул соединений библиотеки `pgx` (листинг **??**).

Пул соединений повторно использует открытые TCP-сессии к PostgreSQL, снижая накладные расходы на постоянную аутентификацию и установление соединения при каждом HTTP-запросе.

```
        httpAddress := ":" + defaultPort

        pool, err := pgx.NewConnPool(pgx.ConnPoolConfig{
                Host:           dbHost,
                Port:           dbPort,
                Database:       dbName,
                User:           dbUser,
                Password:       dbPassword,
                MaxConnections: 10,
        })
        if err != nil {
                slog.Error("cannot connect to postgres", slog.Any("err",
                ↪  err))
                os.Exit(1)
```

Листинг 6 — Инициализация пула соединений `pgx`

3.1.5 Механизм безопасной сборки SQL-запросов

Поскольку имена таблиц и столбцов передаются из веб-интерфейса динамически, прямое использование позиционных плейсхолдеров PostgreSQL (`$1, $2`) для них неприменимо. Для защиты от внедрения вредоносного SQL-кода (SQL Injection) в файле `db.go` реализованы специализированные функции экранирования (листинг **??**).

```
        Name    string
        Columns []string
        Rows    [][]any
}

func quoteIdent(name string) string {
        return `"` + strings.ReplaceAll(name, `"`, `""`) + `"`
```

Листинг 7 — Функции экранирования идентификаторов и литералов

Функция `quoteIdent` экранирует идентификаторы PostgreSQL (имена таблиц, колонок) согласно стандарту: имя оборачивается в двойные кавычки, а внутренние кавычки удваиваются (). Функция `quoteLiteral` обрамляет пользовательские строковые константы в одинарные кавычки с удвоением внутренних одинарных кавычек.

3.2 Интроспекция каталога PostgreSQL и просмотр таблиц

В отличие от статических приложений, где структура таблиц зафиксирована в структурах данных языка программирования, разрабатываемый вебсервис должен уметь взаимодействовать с произвольными таблицами, создаваемыми пользователем в рантайме. Для этого реализована динамическая интроспекция метаданных СУБД.

3.2.1 Чтение системного каталога information_schema

Получение перечня пользовательских таблиц и описания их столбцов выполняется посредством запросов к стандартным системным представлениям `information_schema` СУБД PostgreSQL.

Для получения списка базовых таблиц публичной схемы используется SQL-запрос из файла `get_tables.sql` (листинг **??**).

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public' AND table_type = 'BASE TABLE'
ORDER BY table_name;
```

Листинг 8 — Запрос списка пользовательских таблиц из `information_schema.tables`

Условие `table_schema = 'public'` исключает системные таблицы СУБД, а предикат `table_type = 'BASE TABLE'` фильтрует виртуальные представления (VIEW).

Для чтения структуры выбранной таблицы используется параметризованный запрос из файла `get_columns.sql` (листинг **??**).

```
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_schema = CURRENT_SCHEMA AND table_name = $1
ORDER BY ordinal_position;
```

Листинг 9 — Запрос описания колонок таблицы из `information_schema.columns`

Запрос возвращает имена атрибутов и их типы данных (`data_type`), упорядоченные по физическому положению в таблице (`ordinal_position`).

3.2.2 Стек вызова GET /tables и динамическое сканирование кортежей

Просмотр содержимого таблиц инициируется запросом по маршруту `GET /tables` (для вывода всех таблиц) или `GET /tables/{name}` (для выбранной таблицы). Логика обработки сосредоточена в функции `HandleTables` (листинг **??**).

В строке 10 вызывается функция `newBasePage`, наполняющая навигационный выпадающий список актуальным перечнем таблиц из каталога СУБД. Если конкретная таблица не выбрана, вызывается `getAllTablesData`, опрашивающая каждую таблицу по очереди.

Поскольку схема возвращаемых данных заранее неизвестна, в файле `db.go` реализована функция `scanRowsToTableData` (листинг **??**).

Метод `rows.FieldDescriptions()` извлекает названия колонок из заголовка ответа PostgreSQL, а метод `rows.Values()` считывает кортеж значений в обобщенный слайс `[]any`. Значения даты и времени (`time.Time`) приводятся к стандартизированному строковому виду `"YYYY-MM-DD"`.

Полученная структура `TableData` передается в шаблон `tables.html` (листинг **??**), где выполняется отрисовка HTML-таблицы.

Цикл `{{ range .Columns }}` генерирует заголовки таблицы (`<th>`), а вложенный цикл `{{ range .Rows }}` итерирует строки и ячейки (`<td>`).

```
8
9  func HandleTables(views *ViewRenderer, pool *pgx.ConnPool) AppHandler {
10         return func(w http.ResponseWriter, r *http.Request) error {
11                 base, err := newBasePage(pool)
12                 if err != nil {
13                         return &AppError{Err: err, Message: "Database
                           ↪  error: " + err.Error(), Code:
                           ↪  http.StatusInternalServerError}
14                 }
15
16                 data := TablesPageData{BasePage: base}
17                 tableName := r.PathValue("name")
18                 if tableName == "" {
19                         tableName = r.URL.Query().Get("table")
20                 }
21
22                 if tableName == "" {
23                         data.ShowViewAll = true
24                         if all, err := getAllTablesData(pool); err == nil {
25                                 data.AllTables = all
26                         }
27                 } else {
28                         data.SelectedTable = tableName
29                         td, err := getTableData(pool, tableName)
30                         if err != nil {
31                                 data.Status = "error"
32                                 data.Message = "Table not found: " +
                                   ↪  err.Error()
33                                 return views.RenderStatus(w, "tables",
                                   ↪  http.StatusNotFound, data)
34                         }
35                         data.CurrentTable = td
```

Листинг 10 — Обработчик страницы просмотра таблиц в `handlers_tables.go`

```
103         }
104         return tables, rows.Err()
105 }
106
107 func getColumns(pool *pgx.ConnPool, table string) ([]ColumnInfo, error) {
108         rows, err := pool.Query(getColumnsSQL, table)
109         if err != nil {
110                 return nil, err
111         }
112         defer rows.Close()
113
114         var cols []ColumnInfo
115         for rows.Next() {
116                 var col ColumnInfo
117                 if err := rows.Scan(&col.Name, &col.DataType); err != nil {
118                         return nil, err
119                 }
120                 cols = append(cols, col)
121         }
122         return cols, rows.Err()
123 }
124
125 func scanRowsToTableData(rows *pgx.Rows, tableName string) (*TableData,
    ↪  error) {
126         fields := rows.FieldDescriptions()
127         colNames := make([]string, len(fields))
128         for i, f := range fields {
129                 colNames[i] = f.Name
130         }
131
132         var dataRows [][]any
```

Листинг 11 — Динамическое чтение метаданных выборки и значений кортежей

```
            {{ range .AllTables }}
            <hr>
            <h4>Table <code>{{ .Name }}</code></h4>
            <figure>
                <table>
                    <thead>
                        <tr>
                            {{ range .Columns }}
                            <th>{{ . }}</th>
                            {{ end }}
                        </tr>
                    </thead>
                    <tbody>
                        {{ range .Rows }}
                        <tr>
                            {{ range . }}
                            <td>{{ . }}</td>
                            {{ end }}
                        </tr>
                        {{ else }}
                        <tr><td colspan="{{ len .Columns
                         ↪  }}"><em>Empty</em></td></tr>
```

Листинг 12 — Отрисовка динамической таблицы в `templates/tables.html`

3.3 Реализация команд DDL

Команды языка определения данных (Data Definition Language, DDL) отвечают за создание, модификацию и удаление объектов базы данных. В веб-приложении операции DDL сгруппированы по маршруту `/ddl/{op}` со следующими режимами: `create`, `alter`, `truncate`, `drop`.

3.3.1 Создание таблиц: двухэтапный визард и стек вызова POST /ddl/create

Процесс создания таблицы разделен на два логических шага:

а) **Шаг 1**: пользователь указывает желаемое количество столбцов будущей таблицы;

б) **Шаг 2**: динамически строится форма с полями конфигурации каждого столбца: наименование, тип данных PostgreSQL (`INTEGER`, `TEXT`, `BOOLEAN`, `DATE`), признак допустимости неопределенных значений (`Nullable`), радиокнопка выбора первичного ключа (`PRIMARY KEY`) и параметры внешнего ключа (`FOREIGN KEY`).

Фрагмент формы второго шага приведен в листинге **??**.

При отправке формы запрос поступает на эндпоинт `POST /ddl/create`, где запускается парсер параметров `parseCreateColumns` (листинг **??**).

Особое внимание уделено поддержке самоссылающихся внешних ключей (префикс `__self__`), когда столбец ссылается на первичный ключ этой же создаваемой таблицы (например, поле `manager_id` в таблице сотрудников).

После формирования списка столбцов и ограничений целостности обработчик `CreateTableHandler` формирует итоговый SQL-запрос и передает его СУБД (листинг **??**).

Сформированная команда имеет канонический синтаксис PostgreSQL:

```
CREATE TABLE "имя_таблицы" (
    "колонка_1" ТИП NOT NULL,
    PRIMARY KEY ("колонка_pk")
);
```

```
23          <article>
24              <h3>CREATE TABLE</h3>
25              <form action="/ddl/create" method="post">
26                  <input type="hidden" name="col_count" value="{{
                    ↪  .DDLColCount }}">
27                  <label>
28                      Name:
29                      <input type="text" name="table_name" value="{{
                        ↪  .TableName }}" required autofocus>
30                  </label>
31
32                  <figure>
33                      <table>
34                          <thead>
35                              <tr>
36                                  <th>№</th>
37                                  <th>Name</th>
38                                  <th>Type</th>
39                                  <th>Nullable</th>
40                                  <th>FK</th>
41                                  <th>ON DELETE</th>
```

Листинг 13 — Динамическая форма настройки колонок таблицы в `templates/ddl.html`

В случае успешного исполнения запроса клиент перенаправляется на страницу созданной таблицы `/tables/{name}`. При ошибке (например, дублирование имени таблицы или несовместимость типов) форма отображается повторно с сохранением введенных данных и выводом сообщения сервера СУБД.

3.3.2 Модификация схемы: стек вызова POST /ddl/alter

Операция изменения структуры таблицы реализуется обработчиком `AlterTableHandler` (листинг **??**).

Поддерживаются три сценария:

а) **Добавление нового столбца**: формируется SQL-запрос вида `ALTER TABLE ”table”ADD COLUMN ”col”TYPE;;`

```
53                  if strings.HasPrefix(fkColRaw, "__self__:") {
54                          fkTable = "__self__"
55                          if idx, err :=
                             ↪  strconv.Atoi(strings.TrimPrefix(fkColRaw,
                             ↪  "__self__:")); err == nil {
56                                  fkSelfIdx = idx
57                          }
58                  } else if strings.Contains(fkColRaw, ":") {
59                          parts := strings.SplitN(fkColRaw, ":", 2)
60                          fkTable = parts[0]
61                          fkColRaw = parts[1]
62                  } else if fkSelfStr :=
                     ↪  strings.TrimSpace(r.FormValue(fmt.Sprintf("fk_self_col_%d",
                     ↪  i))); fkSelfStr != "" {
63                          if idx, err := strconv.Atoi(fkSelfStr); err == nil
                             ↪  {
64                                  fkSelfIdx = idx
65                          }
66                  }
67
68                  if fkTable == "__self__" && fkSelfIdx == -1 {
69                          if idx, err := strconv.Atoi(fkColRaw); err == nil {
70                                  fkSelfIdx = idx
71                          }
72                  }
73
```

Листинг 14 — Парсинг полей и первичного ключа в `handlers_ddl.go`

б) **Изменение типа существующего столбца**: формируется запрос с явным приведением через конструкцию `USING col::TYPE`;

в) **Удаление столбца**: формируется запрос `ALTER TABLE "table"DROP COLUMN "col"CASCADE;`. Использование директивы `CASCADE` гарантирует, что PostgreSQL автоматически удалит зависимые индексы и ограничения.

3.3.3 Очистка и удаление таблиц: стек вызовов TRUNCATE и DROP

Для быстрого удаления всех записей таблицы без разрушения ее схемы используется обработчик `TruncateTableHandler` (листинг **??**).

Удаление сущности целиком выполняется обработчиком `DropTableHandler` (листинг **??**).

```
                                d.DDLColCount = colCount
                                d.TableName = tableName
                                d.CreateColumns = parsedCols
                                d.AllTableColumns =
                                ↪  getAllTableColumns(pool, d.Tables)
                        })
                }

                if tableName == "" {
                        return renderErr("Table name cannot be empty", "")
                }
                if len(colDefs) == 0 {
                        return renderErr("At least one column must be
                        ↪  specified", "")
                }

```

Листинг 15 — Сборка и выполнение запроса `CREATE TABLE`

Указание ключевого слова `CASCADE` предотвращает блокировку операции СУБД в случаях, когда на удаляемую таблицу ссылаются внешние ключи других отношений.

```
                                        d.TableColumns, _ =
                                        ↪  getColumns(pool, table)
                                }
                        })
                }

                if table == "" || effectiveColName == "" {
                        return renderErr("Specify table and column name",
                        ↪  "")
                }

                var querySQL string
                switch action {
                case "add":
                        if colType == "" {
                                colType = "TEXT"
                        }
                        querySQL = fmt.Sprintf("ALTER TABLE %s ADD COLUMN
                        ↪  %s %s;", quoteIdent(table),
                        ↪  quoteIdent(effectiveColName), colType)
                case "alter_type":
                        if colType == "" {
                                colType = "TEXT"
                        }
                        querySQL = fmt.Sprintf("ALTER TABLE %s ALTER COLUMN
                        ↪  %s TYPE %s USING %s::%s;", quoteIdent(table),
                        ↪  quoteIdent(effectiveColName), colType,
                        ↪  quoteIdent(effectiveColName), colType)
                case "drop":
```

Листинг 16 — Сборка и исполнение команд ALTER TABLE

```
306                 if err := r.ParseForm(); err != nil {
307                         return &AppError{Err: err, Message: "Invalid form
                            ↪  data", Code: http.StatusBadRequest}
308                 }
309
310                 table := strings.TrimSpace(r.FormValue("table"))
311                 if table == "" {
312                         return renderDDLError(w, views, pool, "truncate",
                            ↪  1, "", "Select a table to truncate", "", nil)
313                 }
314
315                 querySQL := fmt.Sprintf("TRUNCATE TABLE %s CASCADE;",
                    ↪  quoteIdent(table))
316                 if _, err := pool.Exec(querySQL); err != nil {
317                         return renderDDLError(w, views, pool, "truncate",
                            ↪  1, table, fmt.Sprintf("Failed to execute
                            ↪  TRUNCATE: %v", err), querySQL, nil)
318                 }
319
320                 redirectSuccess(w, r, fmt.Sprintf("/tables/%s",
                    ↪  url.PathEscape(table)))
```

Листинг 17 — Сборка и исполнение команды `TRUNCATE TABLE`

```
331                 table := strings.TrimSpace(r.FormValue("table"))
332                 if table == "" {
333                         return renderDDLError(w, views, pool, "drop", 1,
                            ↪  "", "Select a table to drop", "", nil)
334                 }
335
336                 querySQL := fmt.Sprintf("DROP TABLE %s CASCADE;",
                    ↪  quoteIdent(table))
337                 if _, err := pool.Exec(querySQL); err != nil {
338                         return renderDDLError(w, views, pool, "drop", 1,
                            ↪  table, fmt.Sprintf("Failed to execute DROP
                            ↪  TABLE: %v", err), querySQL, nil)
339                 }
340
341                 redirectSuccess(w, r, "/tables")
342                 return nil
343         }
344 }
```

Листинг 18 — Сборка и исполнение команды `DROP TABLE`

3.4 Реализация команд DML

Команды языка манипулирования данными (Data Manipulation Language, DML) предназначены для добавления, изменения и удаления кортежей. В приложении поддерживаются три базовые операции: `INSERT INTO`, `UPDATE` и `DELETE`.

3.4.1 Пакетная вставка записей: стек вызова POST /dml/insert

Добавление кортежей организовано в два шага:

а) **Шаг 1**: пользователь выбирает целевую таблицу из выпадающего списка и указывает количество добавляемых строк (от 1 до 10);

б) **Шаг 2**: сервер генерирует табличную сетку ввода, где каждый столбец соответствует реальному атрибуту таблицы из `information_schema`, а строки нумеруются функцией `seq`.

Фрагмент генерации сетки ввода приведен в листинге **??**.

При отправке данных обработчик `InsertHandler` вызывает функцию `parseInsertRows` (листинг **??**), формирующую списки значений.

Если значение ячейки оставлено пустым, оно преобразуется в ключевое слово `NULL`. Непустые значения экранируются функцией `quoteLiteral`. В результате формируется пакетный SQL-запрос с перечислением нескольких кортежей через запятую (листинг **??**).

Формат формируемого SQL-запроса:

```
INSERT INTO "таблица" ("колонка_1", "колонка_2")
VALUES ('знач1', 'знач2'), ('знач3', NULL);
```

Такой подход минимизирует количество сетевых обращений к СУБД, выполняя вставку всех записей в рамках единой команды.

```
                    <table>
                        <thead>
                            <tr>
                                <th>#</th>
                                {{ range .TableColumns }}
                                <th>
                                    {{ .Name }}<br>
                                    <small>{{ .DataType }}</small>
                                </th>
                                {{ end }}
                            </tr>
                        </thead>
                        <tbody>
                            {{ range $r := seq .DMLRowCount }}
                            <tr>
                                <td><strong>{{ add $r 1 }}</strong></td>
                                {{ range $.TableColumns }}
                                <td>
                                   <input type="text" name="val_{{ $r }}_{{
                                     ↪  .Name }}" value="{{ getRowVal
                                     ↪  $.InsertRows $r .Name }}"
                                     ↪  placeholder="{{ .Name }}">
                                </td>
                                {{ end }}
```

Листинг 19 — Генерация динамической сетки ввода кортежей в `templates/dml.html`

3.4.2 Модификация данных: стек вызова POST /dml/update

Изменение данных реализуется обработчиком `UpdateHandler` (листинг **??**).

Пользователь указывает имя таблицы, целевую колонку, новое строковое значение, а также условие фильтрации. Предикат ограничения конструируется вспомогательной функцией `buildSimpleWhere` (листинг **??**).

Функция распознает специальные унарные операторы `IS NULL` и `IS NOT NULL`, а для бинарных операторов (`=`, `!=`, `>`, `<`, `LIKE`) выполняет корректное экранирование операндов:

```
UPDATE "таблица"
SET "колонка" = 'новое_значение'
WHERE "колонка_фильтра" = 'значение';
```

```
        insertRows := make([]map[string]string, rowCount)
        var rowTuples []string

        for rIdx := 0; rIdx < rowCount; rIdx++ {
                rowMap := make(map[string]string, len(cols))
                var valLiterals []string
                for _, c := range cols {
                        val := r.FormValue(fmt.Sprintf("val_%d_%s", rIdx,
                        ↪  c))
                        rowMap[c] = val
                        if val == "" {
                                valLiterals = append(valLiterals, "NULL")
                        } else {
                                valLiterals = append(valLiterals,
                                ↪  quoteLiteral(val))
                        }
                }
                insertRows[rIdx] = rowMap
                rowTuples = append(rowTuples, "("+strings.Join(valLiterals,
                ↪  ", ")+")")
        }
        return insertRows, rowTuples
```

Листинг 20 — Парсинг многострочных кортежей и экранирование в `handlers_dml.go`

```
                cols, err := getColumns(pool, table)
                if err != nil {
                        return renderDMLError(w, views, pool, "insert", 2,
                        ↪  table, "Failed to get columns: "+err.Error(),
                        ↪  "", nil)
                }

                var targetCols []string
                for _, col := range cols {
                        targetCols = append(targetCols, col.Name)
                }

                var quotedCols []string
                for _, c := range targetCols {
                        quotedCols = append(quotedCols, quoteIdent(c))
                }
```

Листинг 21 — Сборка и выполнение пакетного запроса `INSERT INTO`

```
177                                 d.UpdateSetCol = setCol
178                                 d.UpdateSetVal = setVal
179                                 d.UpdateWhereCol = whereCol
180                                 d.UpdateWhereOp = whereOp
181                                 d.UpdateWhereVal = whereVal
182                         })
183                 }
184
185                 if table == "" || setCol == "" {
186                         return renderErr("Specify table and column to
                            ↪  update", "")
187                 }
188
```

Листинг 22 — Конструирование и выполнение команды `UPDATE`

```
134                 vals, err := rows.Values()
135                 if err != nil {
136                         return nil, err
137                 }
138                 for i, v := range vals {
139                         if t, ok := v.(time.Time); ok {
140                                 vals[i] = t.Format("2006-01-02")
141                         }
142                 }
```

Листинг 23 — Конструирование предиката WHERE в `db.go`

3.4.3 Удаление данных: стек вызова POST /dml/delete

Удаление строк выполняется обработчиком `DeleteHandler` (листинг **??**).

```
231                 }
232
233                 redirectSuccess(w, r, fmt.Sprintf("/tables/%s",
                    ↪  url.PathEscape(table)))
234                 return nil
235         }
236 }
```

Листинг 24 — Конструирование и выполнение команды `DELETE FROM`

Команда собирается с использованием безопасного предиката фильтрации:

```
DELETE FROM "таблица"
WHERE "колонка" = 'критерий';
```

Если условия не заданы, формируется безусловное удаление всех кортежей целевой таблицы.

3.5 Реализация выборок и соединений (DQL)

Язык запросов к данным (Data Query Language, DQL) в приложении представлен двумя ключевыми механизмами: параметризованным конструктором выборок SELECT и построителем реляционных соединений INNER JOIN.

3.5.1 Параметризованная выборка: стек вызова POST /query/select

Конструктор выборки /select позволяет пользователю интерактивно настраивать три компонента SQL-запроса:

а) **Проекция колонок**: чекбоксы inc_<col> определяют, какие именно поля войдут в результирующий набор (если ни один чекбокс не выбран, подставляется символ *);

б) **Предикаты фильтрации WHERE**: активация чекбокса where_en_<col> включает условие по выбранному полю с поддержкой широкого набора операторов (=, <>, <, >, <=, >=, LIKE, ILIKE, IN, NOT IN, IS NULL, IS NOT NULL);

в) **Сортировка ORDER BY**: выбор столбца упорядочивания и направления (ASC или DESC).

Фрагмент формы настройки выборки представлен в листинге **??**.

Обработка запроса выполняется в функции SelectHandler (листинг **??**).

Код корректно обрабатывает операторы поиска по подстроке (LIKE и ILIKE), автоматически обрамляя значение масками %val%, а для операторов принадлежности множеству (IN, NOT IN) разбивает строку по запятым и экранирует каждый элемент:

```
SELECT "col1", "col2"
FROM "table"
WHERE "salary" >= '70000'
ORDER BY "salary" DESC;
```

Результат запроса считывается методом pool.Query(querySQL), оборачивается в структуру TableData и отображается пользователю.

```
    <form action="/query/select" method="post">
        <input type="hidden" name="table" value="{{ .SelectTable }}">

        <h4>WHERE:</h4>
        <figure>
            <table>
                <thead>
                    <tr>
                        <th>NAME</th>
                        <th>Is included</th>
                        <th>Is in WHERE</th>
                        <th>Operation</th>
                        <th>Value</th>
                    </tr>
                </thead>
                <tbody>
                    {{ range .SelectColumns }}
                    <tr>
                        <td><strong>{{ .Name }}</strong> <small>({{
                        ↪  .DataType }})</small></td>
                        <td>
                            <input type="checkbox" name="inc_{{ .Name }}"
                            ↪  value="1" checked>
```

Листинг 25 — Интерактивный конструктор выборки в `templates/select.html`

3.5.2 Внутреннее соединение таблиц: стек вызова POST /query/join

Соединение таблиц по маршруту `/join` демонстрирует фундаментальную реляционную операцию объединения двух сущностей по общему ключу (`INNER JOIN`). В качестве демонстрационной схемы используются таблицы `lab2_employee_ilinov` и `departments`, имеющие смысловую связь по телефонным номерам или идентификаторам.

Обработчик `JoinHandler` выполняет валидацию и собирает итоговый запрос (листинг **??**).

Для предотвращения коллизий одноименных атрибутов исходных таблиц (например, столбцов `id` или `phone_number`) каждому выбранному полю назначается уникальный псевдоним (алиас) через конструкцию `AS "table.column"`.

Итоговый SQL-запрос, отправляемый на исполнение в PostgreSQL:

```
SELECT t1.*, t2.*
FROM "lab2_employee_ilinov" AS t1
INNER JOIN "departments" AS t2
    ON t1."phone_number" = t2."phone_number";
```

Движок PostgreSQL выполняет внутреннее соединение с сопоставлением ключей, а сервер формирует единую результирующую матрицу, отправляемую в браузер.

```
                                case "IS NULL", "IS NOT NULL":
                                        whereParts = append(whereParts,
                                        ↪  fmt.Sprintf("%s %s",
                                        ↪  quoteIdent(col.Name), op))
                                case "IN", "NOT IN":
                                        items := strings.Split(rawVal, ",")
                                        var quotedItems []string
                                        for _, it := range items {
                                                if trimmed :=
                                                ↪  strings.TrimSpace(it);
                                                ↪  trimmed != "" {
                                                        quotedItems =
                                                        ↪  append(quotedItems,
                                                        ↪  quoteLiteral(trimmed))
                                                }
                                        }
                                        if len(quotedItems) > 0 {
                                                whereParts =
                                                ↪  append(whereParts,
                                                ↪  fmt.Sprintf("%s %s
                                                ↪  (%s)",
                                                ↪  quoteIdent(col.Name),
                                                ↪  op,
                                                ↪  strings.Join(quotedItems,
                                                ↪  ", ")))
                                        }
                                case "LIKE", "ILIKE":
                                        whereParts = append(whereParts,
                                        ↪  fmt.Sprintf("%s %s %s",
                                        ↪  quoteIdent(col.Name), op,
                                        ↪  quoteLiteral("%"+rawVal+"%")))
                                default:
                                        whereParts = append(whereParts,
                                        ↪  fmt.Sprintf("%s %s %s",
                                        ↪  quoteIdent(col.Name), op,
                                        ↪  quoteLiteral(rawVal)))
                                }
                        }
                }

                whereClause := ""
                if len(whereParts) > 0 {
                        whereClause = " WHERE " + strings.Join(whereParts,
                        ↪  " AND ")
                }
```

Листинг 26 — Сборка условий фильтрации, сортировки и запроса в `handlers_query.go`

```
                                selectItems = append(selectItems,
                                ↪  fmt.Sprintf("t2.%s AS %s",
                                ↪  quoteIdent(c.Name),
                                ↪  quoteIdent(t2+"."+c.Name)))
                        }
                }
                if len(selectItems) == 0 {
                        selectItems = append(selectItems, "t1.*", "t2.*")
                }

                querySQL := fmt.Sprintf("SELECT %s\nFROM %s AS t1\nINNER
                ↪  JOIN %s AS t2\n    ON t1.%s = t2.%s;",
                        strings.Join(selectItems, ", "),
                        quoteIdent(t1),
                        quoteIdent(t2),
                        quoteIdent(col1),
                        quoteIdent(col2))

                rows, err := pool.Query(querySQL)
                if err != nil {
                        data.Status = "error"
                        data.Message = "Failed to execute INNER JOIN: " +
                        ↪  err.Error()
                        data.SQL = querySQL
```

Листинг 27 — Сборка и исполнение команды `INNER JOIN` в `handlers_query.go`

3.6 Механизм сброса и начальной инициализации схемы

Поскольку лабораторная работа предполагает активное тестирование деструктивных операций DDL (удаление таблиц через `DROP TABLE`, очистка через `TRUNCATE`) и DML (массовое удаление и обновление), в приложении предусмотрен механизм изоляции и быстрого восстановления исходного состояния базы данных.

3.6.1 Начальная инициализация схемы базы данных

При старте сервера функция `main` вызывает процедуру `initDatabaseTables` (листинг **??**).

Исходные скрипты встроены в бинарный исполняемый файл с помощью директивы `//go:embed`. Функция выполняет следующие шаги:

а) Создает таблицы `lab2_employee_ilinov` и `departments` с помощью скрипта `schema.sql` (если они еще не созданы);

б) Выполняет проверку количества строк (`SELECT COUNT(*)`);

в) При отсутствии записей запускает наполнение таблицы сотрудников 20 строками (`seed_employees.sql`) и таблицы отделов 5 строками (`seed_departments.sql`).

3.6.2 Стек вызова POST /reset: каскадный сброс и восстановление

Кнопка `Reset` в шапке веб-интерфейса отправляет запрос на эндпоинт `POST /reset`, активируя обработчик `ResetHandler` (листинг **??**).

Обработчик вызывает функцию `resetDatabaseTables` (листинг **??**).

Функция запрашивает актуальный список всех существующих таблиц схемы `public` и поочередно удаляет каждую из них с помощью SQL-команды с модификатором каскадности:

<MINTED>

```
43 func quoteLiteral(val string) string {
44         return `'` + strings.ReplaceAll(val, `'`, `''`) + `'`
45 }
46
47
48 func initDatabaseTables(pool *pgx.ConnPool) error {
49         if _, err := pool.Exec(schemaSQL); err != nil {
50                 return fmt.Errorf("failed to execute schema SQL: %w", err)
51         }
52
53         var empCount int
54         if err := pool.QueryRow("SELECT COUNT(*) FROM
           ↪  lab2_employee_ilinov").Scan(&empCount); err != nil {
55                 return err
56         }
57
58         if empCount == 0 {
59                 if _, err := pool.Exec(seedEmployeesSQL); err != nil {
60                         return fmt.Errorf("failed to seed employees: %w",
                           ↪  err)
61                 }
62         }
63
64         var deptCount int
65         if err := pool.QueryRow("SELECT COUNT(*) FROM
           ↪  departments").Scan(&deptCount); err != nil {
66                 return err
67         }
68
69         if deptCount == 0 {
70                 if _, err := pool.Exec(seedDepartmentsSQL); err != nil {
```

Листинг 28 — Инициализация таблиц и первичное наполнение данными в `db.go`

После полной очистки пространства имен схемы повторно вызывается процедура `initDatabaseTables`, возвращающая базу данных в эталонное исходное состояние.

```
                return views.Render(w, "tables", data)
        }
}

func ResetHandler(views *ViewRenderer, pool *pgx.ConnPool) AppHandler {
        return func(w http.ResponseWriter, r *http.Request) error {
                if err := resetDatabaseTables(pool); err != nil {
                        base, bErr := newBasePage(pool)
                        if bErr == nil {
                                base.Status = "error"
                                base.Message = "Database reset failed: " +
                                ↪  err.Error()
                                return views.RenderStatus(w, "tables",
                                ↪  http.StatusInternalServerError,
                                ↪  TablesPageData{BasePage: base,
                                ↪  ShowViewAll: true})
                        }
                        return &AppError{Err: err, Message: "Database reset
                        ↪  failed: " + err.Error(), Code:
                        ↪  http.StatusInternalServerError}
```

Листинг 29 — Обработчик сброса базы данных в `handlers_tables.go`

```
                }
        }

        return nil
}

func resetDatabaseTables(pool *pgx.ConnPool) error {
        tables, err := getTables(pool)
        if err != nil {
                return err
        }
```

Листинг 30 — Каскадное удаление всех таблиц и повторная инициализация

3.7 Результаты работы и снимки экрана приложения

Для подтверждения корректности работы разработанного веб-сервера и взаимодействия с СУБД PostgreSQL было проведено комплексное тестирование всех реализованных сценариев.

Снимок экрана страницы просмотра таблиц схемы представлен на рисунке **??**.

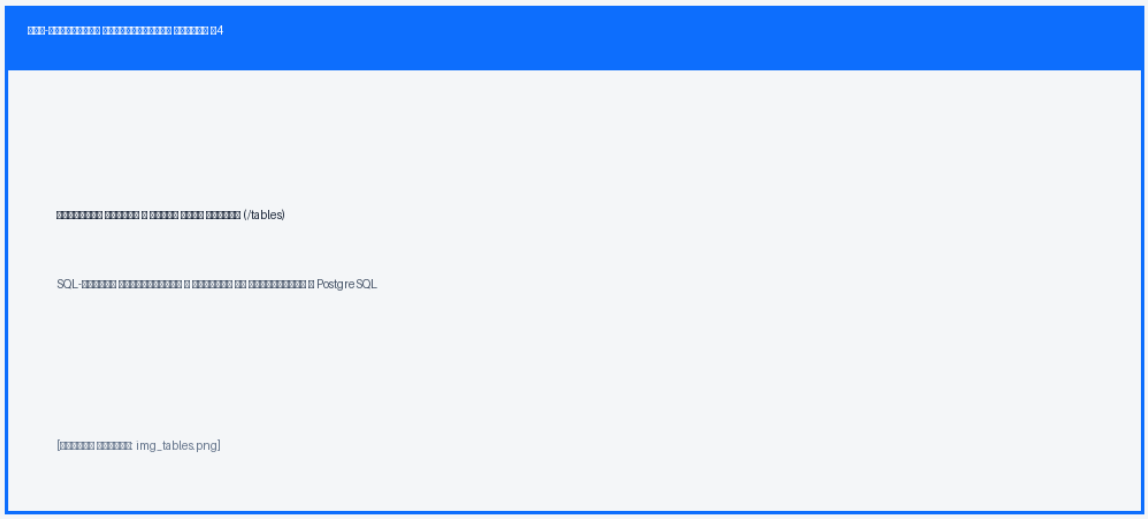


Рисунок 3.2 — Отображение списка таблиц и их кортежей в браузере

Снимок экрана создания новой таблицы с первичным и внешним ключами приведен на рисунке **??**.

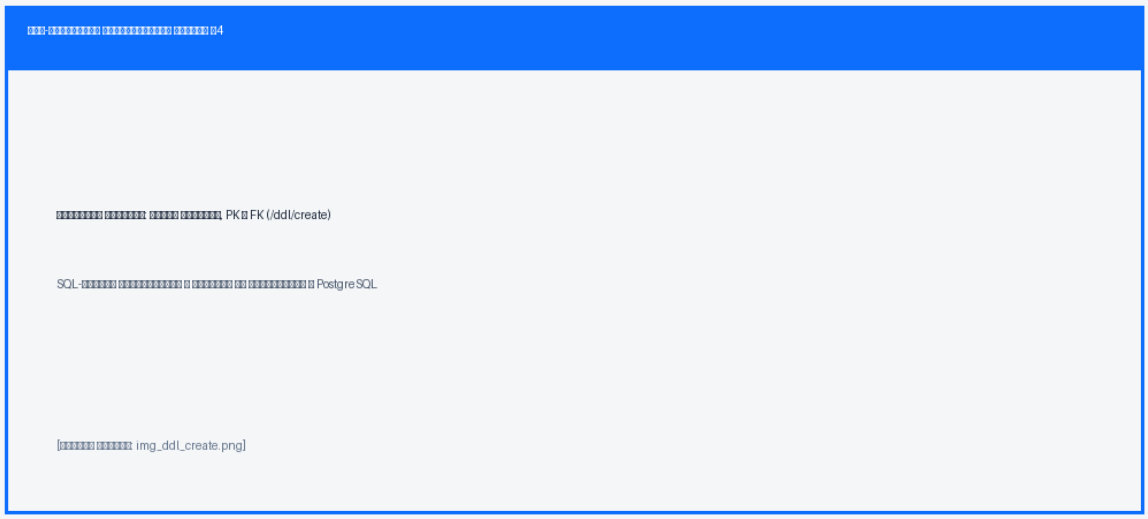


Рисунок 3.3 — Создание таблицы через двухэтапную форму DDL

Снимок экрана модификации колонок таблицы с помощью команды ALTER TABLE приведен на рисунке **??**.

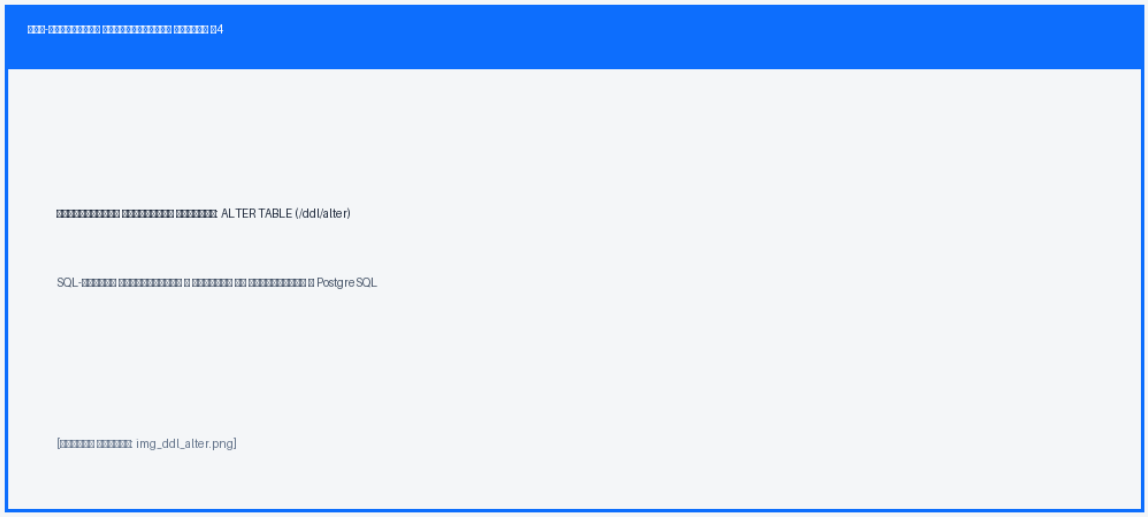


Рисунок 3.4 — Модификация структуры таблицы с помощью команды ALTER TABLE

Снимок экрана очистки данных таблицы оператором TRUNCATE представлен на рисунке **??**.

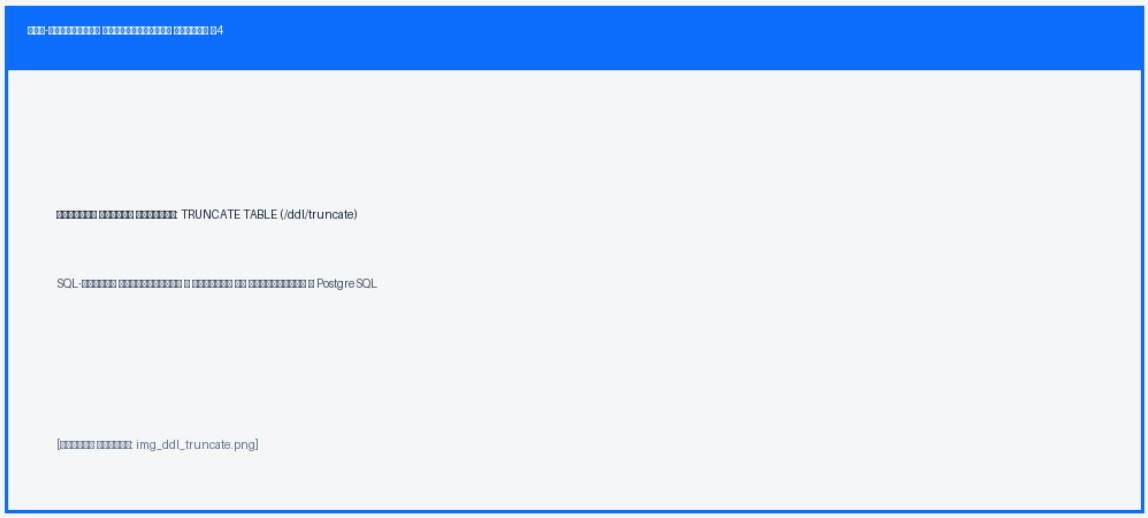


Рисунок 3.5 — Очистка таблицы оператором TRUNCATE TABLE CASCADE

Снимок экрана удаления таблицы с каскадной очисткой связей приведен на рисунке **??**.

Снимок экрана пакетного добавления нескольких кортежей оператором INSERT INTO представлен на рисунке **??**.

Снимок экрана изменения данных оператором UPDATE с предикатом WHERE приведен на рисунке **??**.

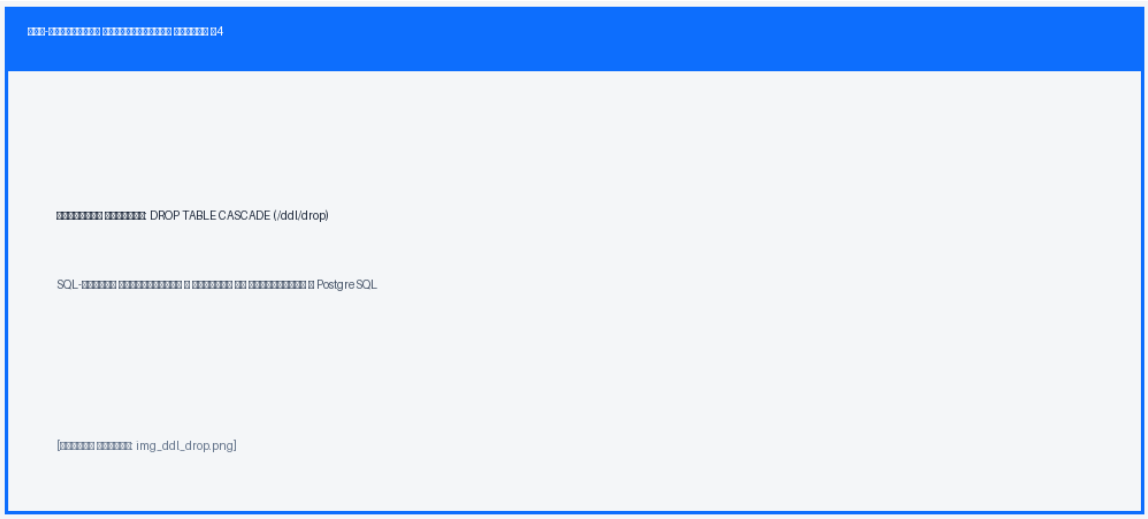


Рисунок 3.6 — Удаление сущности оператором DROP TABLE CASCADE

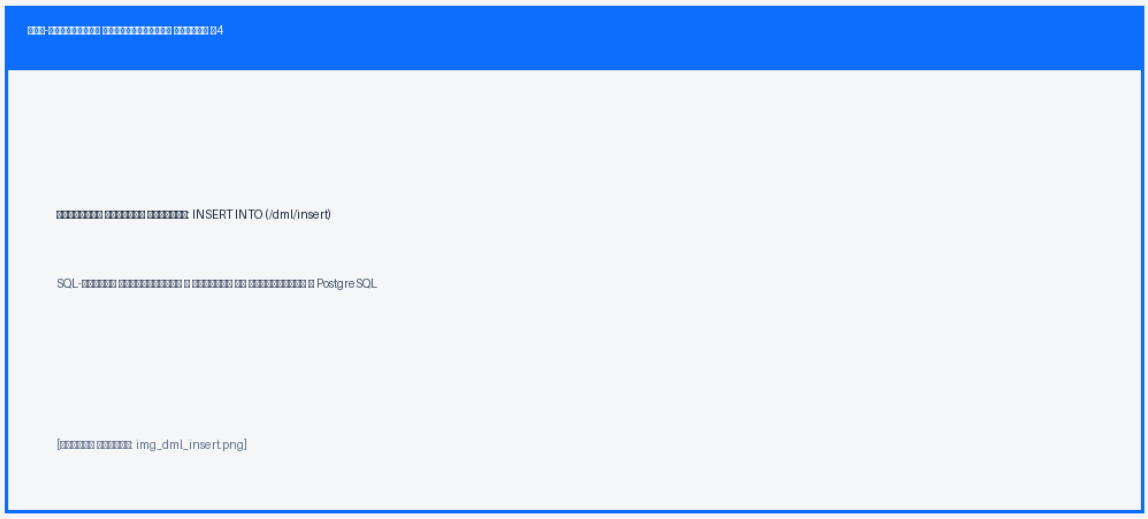


Рисунок 3.7 — Пакетная вставка записей с помощью команды INSERT INTO

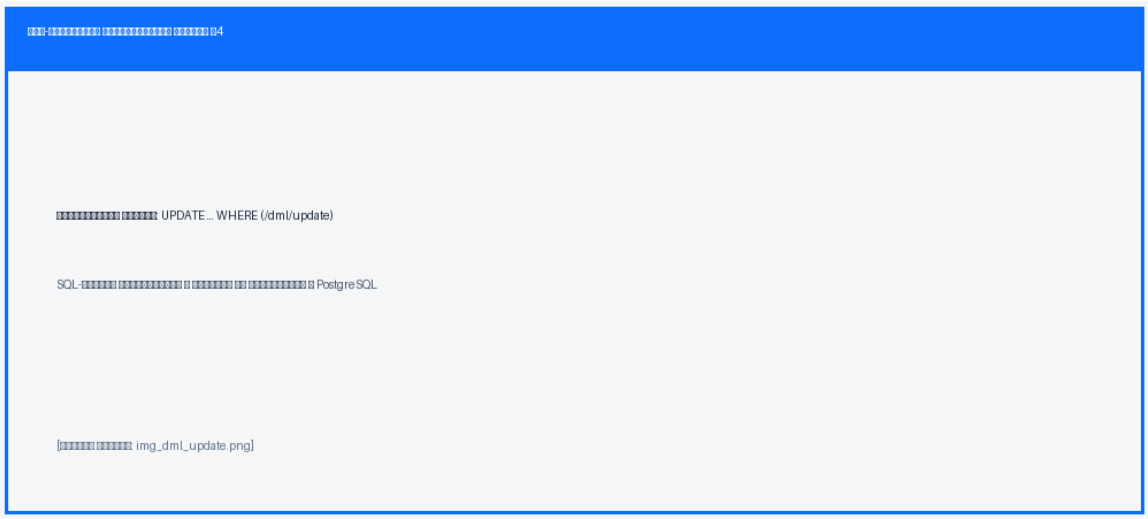


Рисунок 3.8 — Обновление данных с условием фильтрации

Снимок экрана удаления записей оператором DELETE приведен на рисунке **??**.

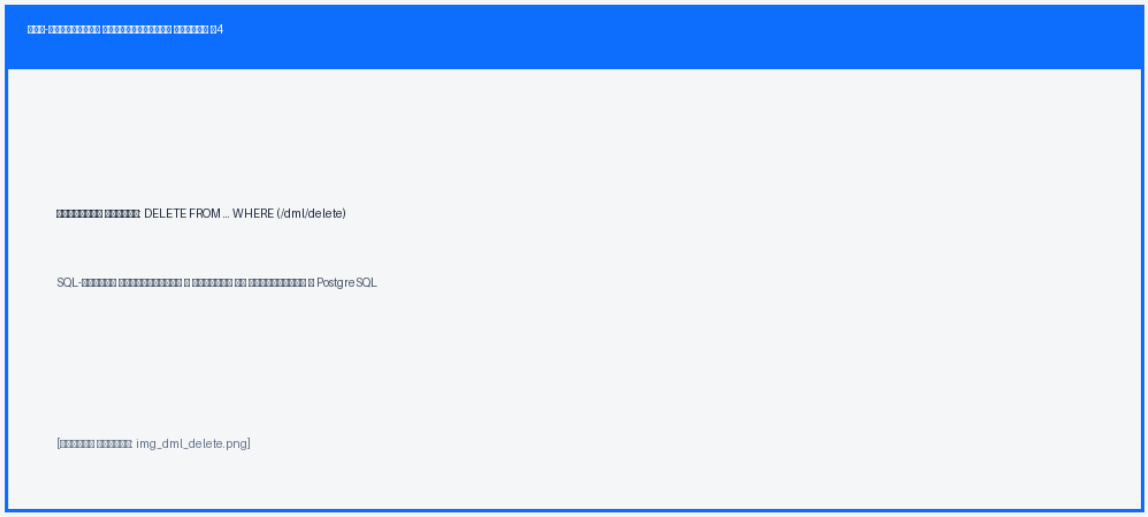


Рисунок 3.9 — Удаление записей оператором DELETE FROM по условию

Снимок экрана параметризованной выборки SELECT с фильтрацией и сортировкой приведен на рисунке **??**.

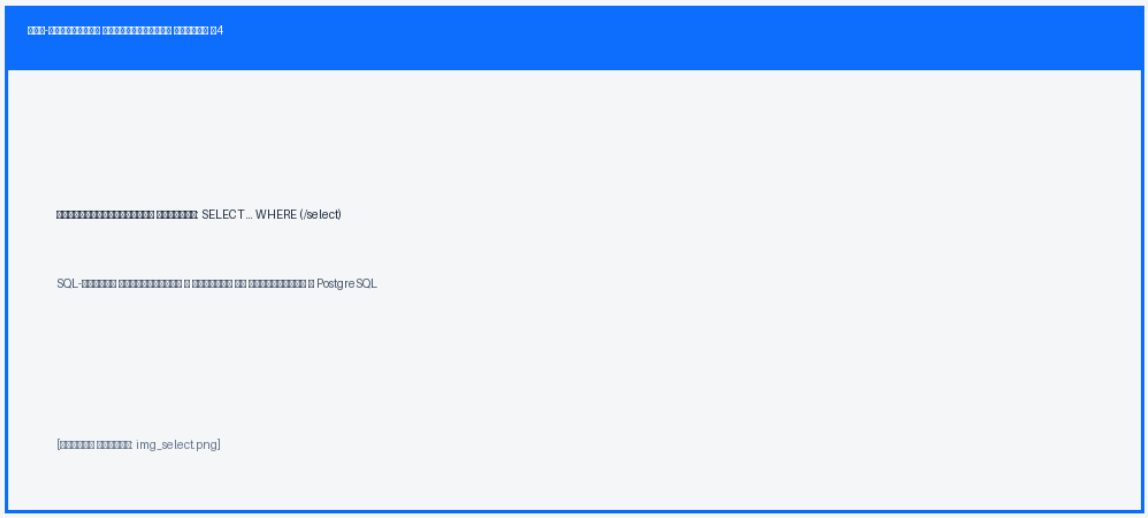


Рисунок 3.10 — Результаты параметризованной выборки SELECT ... WHERE

Снимок экрана внутреннего соединения двух таблиц оператором INNER JOIN представлен на рисунке **??**.

На всех этапах тестирования сформированные SQL-запросы корректно валидировались и исполнялись СУБД PostgreSQL, а результирующие наборы данных и системные уведомления в полном объеме выводились в веб-интерфейсе.

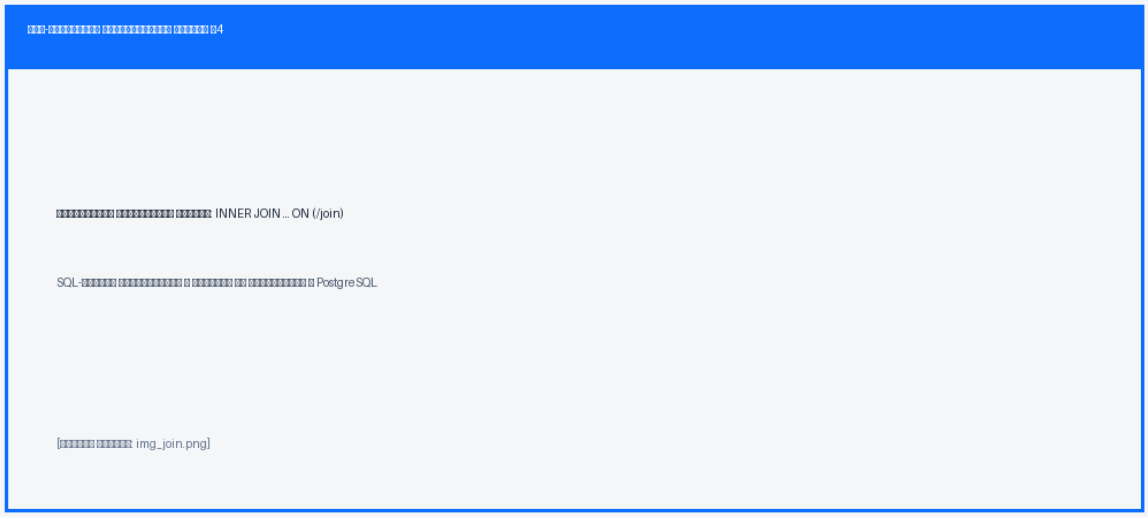


Рисунок 3.11 — Результаты реляционного соединения INNER JOIN ... ON

ЗАКЛЮЧЕНИЕ

В процессе выполнения лабораторной работы были освоены практические навыки работы с языком SQL, реляционной СУБД PostgreSQL и разработкой веб-приложений на языке Golang (версия 1.27) по принципу Server-Side Rendering.

В ходе работы были решены следующие задачи:

а) Разработан веб-сервер с модульной архитектурой и сквозным стеком вызовов от пользовательских HTML-форм до ядра СУБД PostgreSQL;

б) Реализовано безопасное конструирование SQL-запросов с экранированием идентификаторов (`quoteIdent`) и литералов (`quoteLiteral`);

в) Реализована поддержка операций языка определения данных DDL: двухэтапное создание таблиц с поддержкой первичных и внешних ключей (включая самоссылающиеся ограничения), изменение структуры через ALTER TABLE, очистка через TRUNCATE и каскадное удаление через DROP TABLE CASCADE;

г) Реализована поддержка операций языка манипулирования данными DML: пакетная вставка записей с автоматической обработкой NULL-значений, изменение данных по условию UPDATE и удаление данных DELETE;

д) Реализован интерактивный конструктор запросов выборки SELECT с динамической проекцией, фильтрацией и сортировкой, а также оператор реляционного соединения INNER JOIN двух таблиц по общему ключу;

е) Настроена интроспекция метаданных через системные представления каталога `information_schema.tables` и `information_schema.columns`;

ж) Реализован механизм изоляции и быстрого восстановления исходного состояния базы данных через эндпоинт `POST /reset`.

Корректность функционирования подтверждена серией тестов в браузере с прозрачным аудитом текста сформированных SQL-команд.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЕ SQL-СКРИПТЫ ИНИЦИАЛИЗАЦИИ БАЗЫ ДАННЫХ

В данном приложении приведены скрипты создания схемы и наполнения базы данных демонстрационными наборами данных.

Скрипт создания структуры базовых таблиц `lab2_employee_ilinov` и `departments` приведен в листинге **??**.

<MINTED>

Листинг 31 — Скрипт создания таблиц `schema.sql`

Скрипт наполнения таблицы отделов `departments` приведен в листинге **??**.

<MINTED>

Листинг 32 — Скрипт наполнения таблицы `departments`

Скрипт наполнения таблицы сотрудников `lab2_employee_ilinov` приведен в листинге **??**.

<MINTED>

Листинг 33 — Скрипт наполнения таблицы `lab2_employee_ilinov`